



Final Defense Presentation

AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics

Presented By

Md. Riaz

Program: B.Sc. in CSE (Diploma)

ID: 0322310105101024

Batch: 16th

8th Semester / 4th Year

Session: Spring - 2023

Supervised By

Mst. Sahela Rahman

Lecturer

Dept. of CSE, PUB

Department of Computer Science & Engineering
Pundra University of Science & Technology, Bogura - 5800



Outline

1. Introduction and Problem Statement
2. The Idea in One Picture
3. Literature Review and Research Gap
4. Objectives and Methodology
5. AEGIS Architecture (Stages 1-7)
6. Semantic Layer and Compiler
7. Query-to-SQL Walkthrough, Including Joins
8. Prototype Implementation
- 9. Evaluation:** 500 Questions and 20 Admin Reports
10. Limitations, Conclusion, and Future Work
- 11. Appendix:** Schema, Semantic Layer, Compiler Code



Introduction

Motivation:

- E-commerce systems store rich operational data, but built-in dashboards answer only a fixed set of questions.
- When users need new analytical combinations, the request usually depends on a developer or BI person to translate it into report logic.
- Natural language analytics can reduce that delay for managers, sales teams, and administrators.

Central Tension:

- Direct LLM-to-SQL systems are flexible, but the model writes executable database code.
- A safe analytics system should understand the question without giving the model authority to create arbitrary SQL.

AEGIS Direction:

- Use the LLM for intent extraction only.
- Use deterministic semantic mapping and SQL compilation for execution.



Problem Statement

Current Generative NL2SQL approaches have 3 structural vulnerability classes:

1. Structural Injection Risk

Adversarial prompt manipulation can bypass model instructions, causing neural models to output data-modifying DML/DDL statements (DROP, DELETE, UPDATE).

2. Unbounded Schema Hallucination

Probabilistic token generation leads to hallucinated table joins, non-existent entity relations, and invalid column attributes.

3. Access Control & Context Bypass

Direct query generation bypasses application-level multi-tenant boundaries and row-level security scopes.



The Idea in One Picture

In a restaurant, the waiter writes down what you want — the dish, how many plates. The waiter never walks into the kitchen and cooks it.



In the restaurant	In AEGIS	What it cannot do
The waiter	The language model	Cannot write or run SQL
The order form	The Intent Object	Cannot hold a table or column name of its own
The kitchen	The SQL compiler	Cannot accept anything outside the approved list

In plain terms: The model chooses names from an approved list. It never writes the database query itself.



Literature Review

Work	Main focus	Control approach	Gap for AEGIS
NaLIR [1]	Interactive NLIDB	Parse and refine intent with grammar and DB mapping	Limited dashboard and modern LLM risk handling
nl4dv [2]	NL to visualization specs	Extract analytic attributes; not SQL-centered	Visualization focus, not safe database execution
DashBot [3]	Dashboard generation	Insight-driven dashboard selection	Does not solve arbitrary SQL authority risk
PICARD [4]	Constrained decoding	Parser-level SQL validity during token generation	Model still generates SQL text
G-SQL [5] / TriSQL [6]	Robust Text-to-SQL	Schema-aware generation with rules, repair, and refinement	Safety depends on controlling generated SQL



Research Gap

Observed Gap in Existing Systems:

- Many systems improve SQL generation accuracy, but still let executable SQL be authored by the model.
- Visualization systems help users express analysis, but do not fully address database execution safety.
- Parser and repair methods can reject invalid syntax, yet syntactically valid SQL can still be unsafe or semantically wrong.

AEGIS Research Gap:

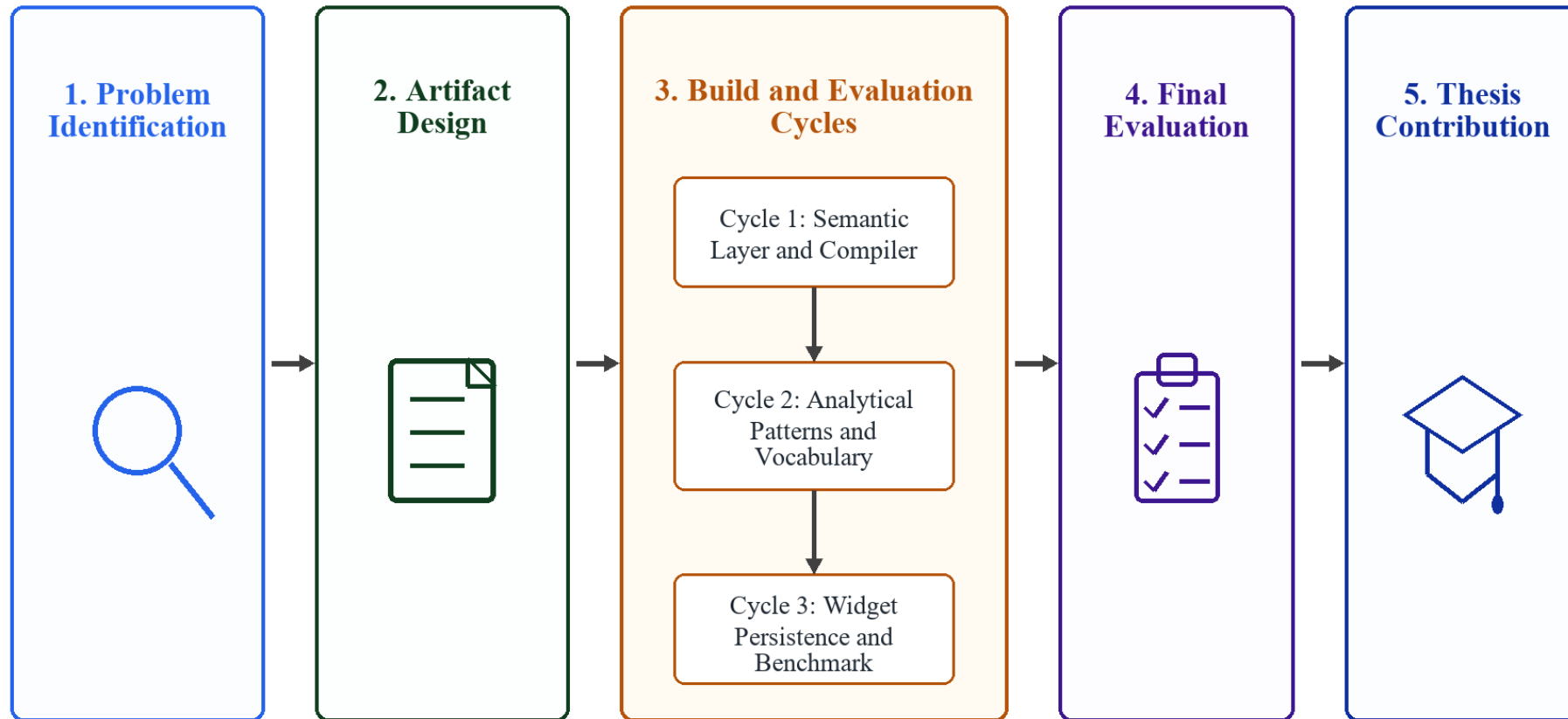
- A practical architecture is needed where natural language is accepted, but executable SQL is produced only from an explicit semantic layer and deterministic compiler.



Objectives and Contributions

Objective	Contribution
Reduce execution risk	Separate natural-language understanding from SQL creation.
Make analytics auditable	Define metrics, dimensions, filters, join paths, and output shapes in a semantic layer.
Support real e-commerce reports	Implement AEGIS over a seeded nopCommerce [7] MySQL database.
Evaluate against an independent reference	Measure coverage on 500 fixed natural-language questions, and accuracy against nopCommerce's own twenty admin reports.

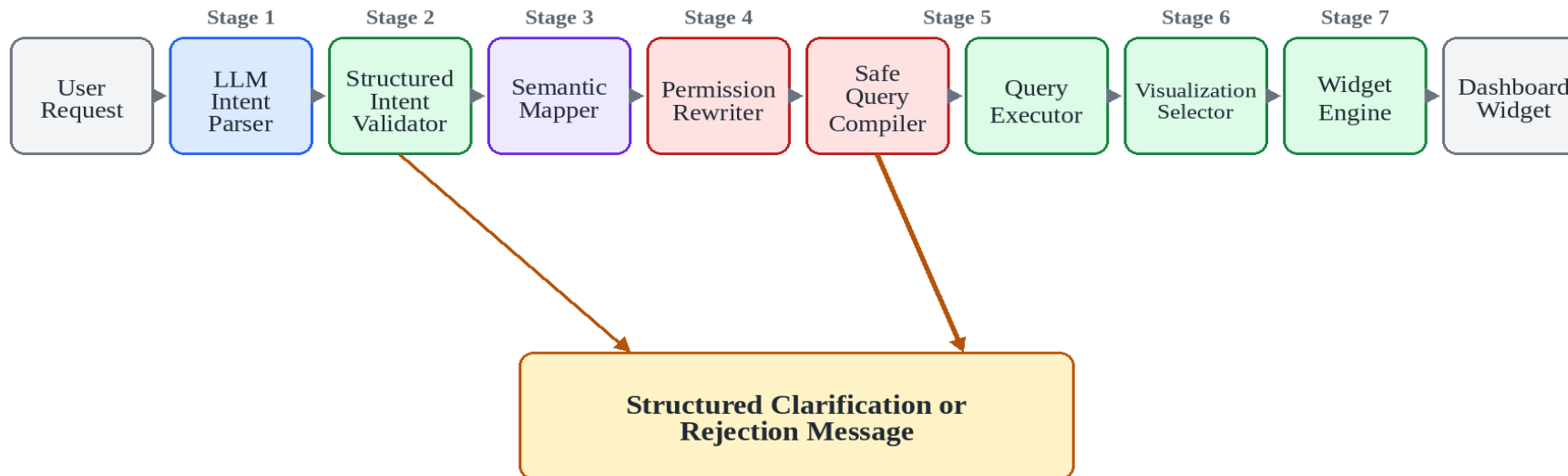
Research Methodology





Proposed AEGIS Architecture

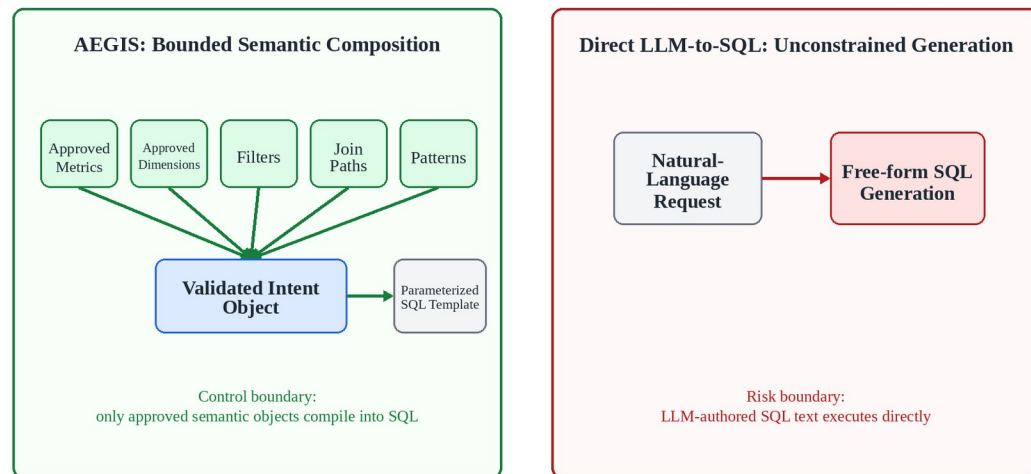
AEGIS Architecture Pipeline



Seven stages. Only Stage 1 uses an LLM; Stages 2-7 are deterministic code. Stage 5 covers both compilation and read-only execution.

Semantic Layer Implementation

Semantic Layer Modularity



Implemented per target system:

- **Business vocabulary:** synonyms for metrics, dimensions, filters, and time periods.
- **Metric definitions:** SQL expressions such as order count, revenue, average order value, and customer count.
- **Dimension definitions:** status, customer, product, manufacturer, category, date, and geography.
- **Join policy:** allowed tables and join paths from the nopCommerce [7] schema.
- **Output contracts:** table, matrix, KPI, and chart-ready shapes.



Example:

Show order average by status

Vocabulary excerpt:

- **Metrics:** revenue, order_count, avg_order_value
- **Dimensions:** order_status, category, country
- **Output:** kpi, table, matrix, chart

```
{
  "intent_class": "segment",
  "metric_term": "avg_order_value",
  "dimension_term": "order_status",
  "output_shape": "table"
}
```

Stage	Allowed	Not allowed
LLM	Choose approved ids from vocabulary	Raw SQL, joins, table names, writes
AEGIS	Bind ids to semantic definitions	Unknown ids, silent fallback, unsafe execution

In plain terms: The model picks names from a list we wrote. It never sees a table name, and never writes SQL.



The Intent Object, and What It Selects

Stages 1 and 2 of 7

IntentObject (aegis/server/models.py)

```
class IntentObject(BaseModel):
    intent_class: IntentClass # required, 11-enum
    metric_term: Optional[str] # metric id
    dimension_term: Optional[str] # dimension id
    time_term: Optional[str] # phrase, not date
    filters: List[Filter] # field/op/value
    limit: Optional[int]
    confidence: Confidence=LOW # absence != high
    unmapped_terms: List[str] # unmatched words

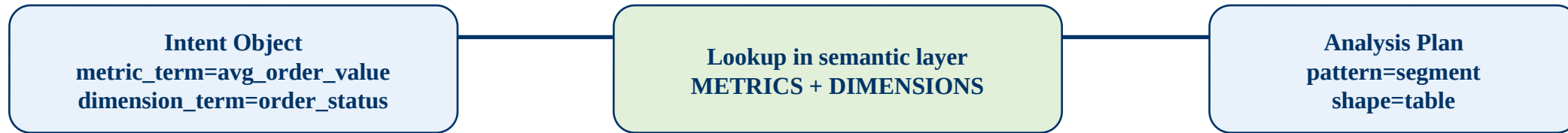
# No sql field. The type cannot express one.
```

intent_class selects the compilation path

intent_class	Compiler path taken
kpi, summary	aggregate, no LIMIT
segment, ranking, comparison, cohort	aggregate + GROUP BY + ORDER BY
trend	date bucket; zero-filled when daily
tabular, exception	row listing, no aggregation
funnel, correlate	aggregate over the stated relation

An id the semantic layer does not define is rejected here, before any SQL exists. There is no path from an unknown id to a query.

In plain terms: The answer form is chosen from eleven fixed shapes, not invented per question.



Slot	Resolved id	Business meaning	SQL expression	Binding
Metric	avg_order_value	Average revenue per order	AVG(COALESCE(o.OrderTotal, 0))	Order as o
Dimension	order_status	Readable order status label	CASE o.OrderStatusId ... END	Order as o
Rule	mandatory	Ignore soft-deleted orders	o.Deleted = 0	WHERE

Intent Object vs Analysis Plan: the Intent Object is what the model said. The Analysis Plan is what AEGIS was able to bind it to — it adds the pattern, the join path, the resolved SQL expressions, and the evidence for each binding. Only the Plan reaches the compiler.

In plain terms: The model said 'average order value'. The definition behind that name was written by the administrator.

Template-Based SQL Compilation

Compiler template

```
SELECT {dimension_expr} AS label,  
       {metric_expr} AS value  
FROM {base_table}  
{join_clauses}  
WHERE {mandatory_predicates}  
GROUP BY {dimension_group_expr}  
ORDER BY value DESC;
```

Filled from semantic layer

- **dimension_expr**: CASE order status mapping
- **metric_expr**: AVG(order total)
- **base_table**: Order o
- **joins**: none required here
- **predicates**: o.Deleted = 0

Compiled safe SELECT

```
SELECT CASE o.OrderStatusId  
        WHEN 10 THEN "Pending"  
        WHEN 20 THEN "Processing"  
        WHEN 30 THEN "Complete"  
        WHEN 40 THEN "Cancelled"  
        END AS label,  
        AVG(COALESCE(o.OrderTotal, 0)) AS value  
FROM `Order` o  
WHERE o.Deleted = 0  
GROUP BY CASE o.OrderStatusId ... END  
ORDER BY value DESC;
```

The template controls SQL structure; the semantic layer supplies approved identifiers and expressions; user text is never inserted into the query.

In plain terms: The query's shape is fixed in advance. Only approved names are dropped into the blanks.



Walkthrough: A Question That Needs Four Joins

Stages 1 to 5 of 7

NQ257 "Show product revenue by category."

Intent Object from the model

```
{
  "intent_class": "segment",
  "metric_term": "line_item_revenue",
  "dimension_term": "category_name"
}
```

Join path resolved by BFS, not by the model

```
Product -> OrderItem -> Order
        -> Product_Category_Mapping
        -> Category
```

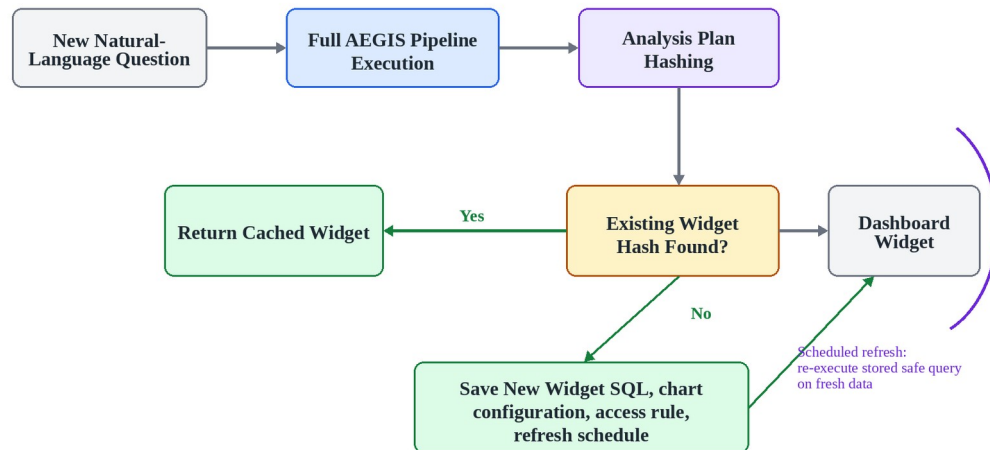
Compiled SQL, executed read-only

```
SELECT c.Name AS label, SUM(oi.PriceExclTax) AS value
FROM `Order` o
LEFT JOIN `OrderItem` oi ON o.Id = oi.OrderId
LEFT JOIN `Product` p ON oi.ProductId = p.Id
LEFT JOIN `Product_Category_Mapping` pcm ON p.Id = pcm.ProductId
LEFT JOIN `Category` c ON pcm.CategoryId = c.Id
WHERE 1=1
      AND o.Deleted = 0
      AND p.Deleted = 0
GROUP BY c.Name
ORDER BY value DESC
LIMIT 100
```

Also worth seeing	What actually happens	Why it matters
"stock less than 10"	compiles to <code>p.StockQuantity < @p0</code> with params {p0: 10}	The number is bound, never pasted into the SQL text
"revenue from Google Ads"	declined: traffic_source is not in the semantic layer	No guess, no nearest-match substitution

Output Generation

Widget Lifecycle and Refresh Model



Output is based on result shape, not model narration:

- **KPI:** one numeric value with label.
- **Table:** row/column data with responsive scrolling in the prototype.
- **Matrix:** period columns such as today, week, month, year, and all time.
- **Chart-ready output:** categories and values are produced from query results.

This lets the prototype show both the answer and the evidence path used to produce it.



Prototype Implementation

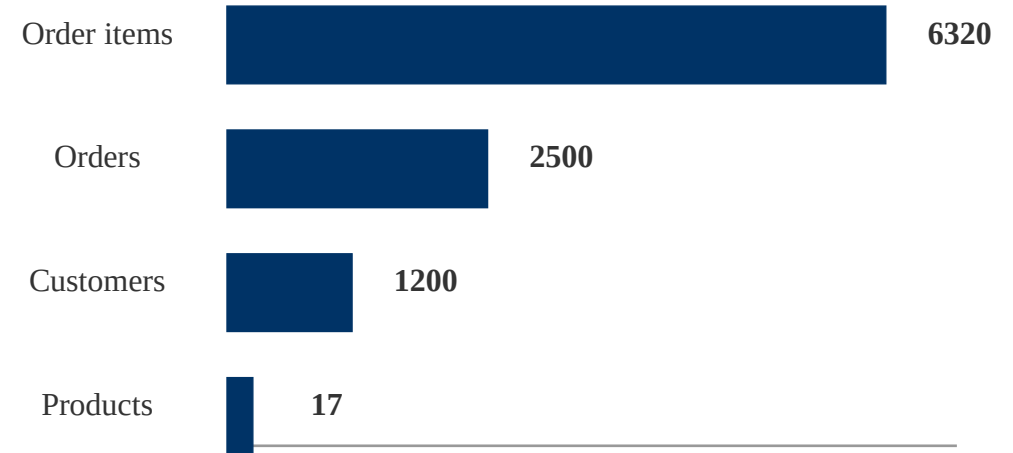
Prototype stack:

- FastAPI backend for NL analytics requests.
- MySQL seeded with nopCommerce-style [7] commerce data.
- OpenAI-compatible LLM API for intent extraction.
- Semantic-layer files define metrics, dimensions, filters, and output shapes.
- Frontend shows answer output plus AEGIS stage evidence.

Reproducibility:

- Docker build includes app, database seed scripts, and smoke checks.
- Local health test verified database connection and seeded counts.

Seeded Data Used in Live Prototype





Evaluation Track 1: 500 Natural-Language Questions

425 questions the semantic layer should answer, 75 realistic e-commerce questions it should decline. The same question set is run two ways, and they answer different questions.

Measure	Deterministic run (no model in the loop)	End-to-end run (live parser)
Parser produced an intent	not applicable	499/500 (99.8%)
Supported questions answered	425/425 (100.0%)	423/425 (99.5%)
Compiled SQL executed	425/425 (100.0%)	422/425 (99.3%)
Intent matched the expected answer	not applicable	313/425 (73.6%)
Out-of-scope questions declined	75/75 (100.0%)	72/75 (96.0%)

The left column has no model in it: saved expected intents are fed straight to the mapper, so it tests the compiler, not the pipeline. Only the right column is an end-to-end result, and it is the one to quote.

Declining is only meaningful next to answering: a system that refuses everything scores 100% on the last row.

In plain terms: It answers the questions it was built for, and says no to the ones it was not — on purpose.



Where the Time Goes

Median and 95th-percentile time per question, measured on the supported questions in the 500-question set.

Stage	Who runs it	Median	95th percentile
1. Intent extraction	The language model	9,029.89 ms	19,460.15 ms
2. Semantic resolution	AEGIS	0.89 ms	1.25 ms
3. SQL compilation	AEGIS	0.13 ms	0.21 ms
4. Database execution	MySQL	2.54 ms	12.87 ms
Stages 2-4 combined	Everything after the model	3.62 ms	14.02 ms

Almost all of the wall clock is the model reading the question. The part that decides which tables to join, writes the SQL, and runs it takes a few milliseconds — and the model has no influence over any of it.

The model figure is a property of the gateway used here, not of the architecture; the deterministic figures are the ones that would follow AEGIS to another deployment.



Without the Semantic Layer: the Same Model, Unconstrained

The identical model through the identical gateway, asked to write MySQL directly for the same 500 questions.

Measure	AEGIS	Direct LLM-to-SQL
Supported questions whose SQL executed	422/425 (99.3%)	365/425 (85.9%)
Out-of-scope questions answered anyway	3/75 (4.0%)	25/75 (33.3%)
Queries containing a forbidden construct	0 — the compiler cannot emit one	2/500 (0.4%)

What answering an unanswerable question looks like

The question	What the unconstrained model returned
"Forecast next month's total sales."	A query summing past months. It runs, returns a number, and forecasts nothing.
"Which customers are likely to churn?"	A query counting each customer's past orders. No prediction is possible from it.
"Summarize the top complaints in reviews."	Keyword matching over review text, joined with UNION — rejected by the safety scan.

In plain terms: Asked something the data cannot answer, the unconstrained model answers anyway — and the answer looks right.



Evaluation Track 2: nopCommerce's Own 20 Admin Reports

The report list is nopCommerce's own admin menu, and the comparison target is nopCommerce's own report code, read from source at commit 64bdf2ff. Neither was chosen by this project.

Check	Result	What it does and does not show
Report reached an answer and compiled to SQL	20/20	Coverage of the report shape. Not correctness — several once passed this while silently wrong.
Result set matched the platform's own query	15/20 (75.0%)	The check that tests the claim: same database, same rows, same values.

The five reports that did not match, and why

Difference	Reports affected
Row count only — the platform's report carries its own LIMIT (5, 15, 100)	Bestsellers by quantity, Bestsellers by amount, Latest orders, Best customers by order count
Label column — customer name where the platform's query labels by email	Best customers by order total, Best customers by order count
Wrong value	None. Every overlapping row agrees to the last decimal.

In plain terms: Where the numbers differ from nopCommerce's own reports, they do not — only how many rows and which label.



Beneficiaries and Expected Impact

Beneficiary	Current difficulty	AEGIS impact
Business users	Need reports but need new report combinations	Ask natural-language analytics questions and receive controlled table or chart-ready results
Developers / BI teams	Repeated report requests consume implementation time	Reusable semantic definitions reduce one-off SQL writing for common analytics
Database administrators	Model-written SQL is hard to audit and restrict	Only approved metrics, dimensions, joins, and read-only SQL reach the database
Researchers	NL2SQL work often mixes language accuracy with execution authority	Shows an architecture where language understanding and SQL authority are separated

Expected impact: safer self-service analytics for repeated e-commerce reporting, without giving the LLM permission to author executable SQL.



Scope of the Evaluation

Current evaluation scope:

- Implemented and evaluated for nopCommerce [7] analytics on MySQL; the semantic layer is deliberately finite and deployment-specific.
- **The layer is per-deployment, so cross-domain benchmarks such as Spider and BIRD do not apply:** they test generalisation to unseen schemas, which this architecture does not attempt.
- The gateway resolves a model alias per request, so the run is not pinned to one model. Each result row records what served it.

Not a thesis limitation:

- Multi-turn conversation is out of scope because the work evaluates single-request analytics.



Limitations I Would Raise First

- Correctness at scale is measured on the 20 reports, not on all 425 supported questions. For those I show the SQL runs — not that every answer is right.
- **The seeded catalogue is small:** 17 products in 8 categories against 2,500 orders. Order-side reports run at realistic volume, but bestsellers and revenue-by-category agree with nopCommerce over short lists — same compiled SQL, weaker test.
- **Three of the 75 boundary questions were not declined:** one malformed model reply, and the same carrier-comparison question phrased twice. The model flagged "DHL" and "FedEx" as unmapped and AEGIS answered by shipping method anyway — by design, because a model-reported gap is evidence, not a verdict. That keeps false refusals low and costs exactly this.
- Five reports differ from the platform's own output in row count or label column. No report differs in value.
- Other SQL dialects require compiler extensions.



Conclusion

Conclusion:

- AEGIS keeps the LLM useful for language understanding while removing SQL-authoring authority from the model.
- The semantic layer makes analytics definitions explicit, auditable, and reusable for a target system.
- The compiler produces read-only SQL from approved definitions and blocks unsupported execution paths.
- The nopCommerce [7] prototype, the 500-question set, and the row-by-row comparison against the platform's own reports show that the approach can be implemented and evaluated practically.

Main contribution:

- A constraint-based architecture for safe LLM-assisted natural language analytics.



Future Work

Possible extensions:

- Evaluate AEGIS on another e-commerce system by replacing only the semantic layer.
- Add more reference reports from real admin and business workflows.
- Extend compiler modules for PostgreSQL and SQL Server.
- Improve clarification behavior for ambiguous but answerable user questions.
- Add richer chart recommendation while keeping SQL generation deterministic.



References

- [1] Li, F., & Jagadish, H. V. (2014). Constructing an interactive natural language interface for relational databases. PVLDB.
- [2] Narechania, A., Srinivasan, A., & Stasko, J. (2021). nl4dv: A toolkit for generating analytic specifications from natural language. IEEE TVCG.
- [3] Deng, D., Wu, A., Qu, H., & Wu, Y. (2023). DashBot: Insight-driven dashboard generation. IEEE TVCG.
- [4] Scholak, T., Schucher, N., & Bahdanau, D. (2021). PICARD: Parsing incrementally for constrained auto-regressive decoding. EMNLP.
- [5] Shalaan, H. S. et al. (2025). G-SQL: A schema-aware and rule-guided approach for robust natural language to SQL translation. IEEE Access.
- [6] Su, X. et al. (2026). A robust natural language text-to-SQL generation framework with dynamic strategies based on LLMs. Scientific Reports.
- [7] nopSolutions. nopCommerce open-source e-commerce platform. GitHub repository.



Thank You & Discussion

THANK YOU!

Questions & Final Defense Discussion



Appendix A: The Semantic Layer Is a File I Wrote

aegis/server/semantic_layer.py — two of the objects the model is allowed to name:

```
Metric(  
    id="avg_order_value",  
    label="Average Order Value",  
    description="Average revenue per order",  
    sql_expr="AVG(COALESCE(o.OrderTotal, 0))",  
    binding_table="Order",  
    required_joins=[],  
    default_visual="kpi_card",  
    time_anchor="o.CreatedOnUtc",  
)
```

```
Dimension(  
    id="order_status",  
    label="Order Status",  
    sql_expr="CASE o.OrderStatusId "  
        "WHEN 10 THEN 'Pending' "  
        "WHEN 20 THEN 'Processing' "  
        "WHEN 30 THEN 'Complete' "  
        "WHEN 40 THEN 'Cancelled' "  
        "ELSE 'Unknown' END",  
    binding_table="Order",  
    datatype="string",  
)
```

The model may return the string "avg_order_value". It cannot return the expression, the table, or the join. Deploying AEGIS on another system means writing this file again — and nothing else.

In plain terms: This list is the whole vocabulary. Anything not written here cannot be asked for.



Appendix B: The Compiler Fills Blanks, It Does Not Write SQL

aegis/server/compiler.py — the function that builds the SELECT clause:

```
def _assemble_select(self, metric, dimension, extra_metrics=None, plan=None) -> str:
    measures = [f"{metric.sql_expr} AS value"]
    for extra in extra_metrics or []:
        measures.append(f"{extra.sql_expr} AS `{extra.id}`")
    projected = ", ".join(measures)

    if dimension:
        dim_expr = self._dimension_select_expr(dimension, plan)
        return f"SELECT {dim_expr} AS label, {projected}"
    return f"SELECT {projected}"

# Every interpolated value is an attribute of a semantic-layer object.
# No argument to this function carries user text.
```

Literal values take a separate route: a filter compiles to a placeholder and a bound parameter (p.StockQuantity < @p0 with {p0: 10}), so a value never becomes part of the SQL text. After assembly, a forbidden-pattern scan rejects any non-SELECT construct before execution.



Appendix C: Why Templates Instead of Constrained Decoding

	PICARD [4] — constrained decoding	AEGIS — template compilation
What the model emits	SQL text, token by token	A typed Intent Object with no SQL field
What is checked	Each token against a parser, during generation	Each id against the semantic layer, before compilation
What we give up	Nothing — any valid SQL stays reachable	Flexibility: a question outside the semantic layer cannot be answered at all

Constrained decoding guarantees the SQL parses. It does not guarantee the SQL is the query the administrator would have approved — a syntactically perfect query can still join the wrong grain, drop a soft-delete filter, or read a column nobody meant to expose.

The trade is real and worth stating plainly: AEGIS buys auditability by giving up open-ended coverage. That is why 75 of the 500 evaluation questions are ones it must decline.